FUTURE INTERNS — CYBER SECURITY INTERNSHIP (TASK 3)

API Security Risk Analysis Report

OWASP API Security Top 10 (2023) Audit & Vulnerability Assessment for SaaS APIs

TARGET PLATFORM
ReqRes API (https://reqres.in)

ASSESSMENT FRAMEWORK
OWASP API Security Top 10

TESTING MODE
Passive & Non-Intrusive (Read-Only)

LEAD ASSESSOR
Harsha H (AppSec Intern)

1. Executive Summary

Modern applications rely predominantly on REST and GraphQL APIs to power mobile clients, web dashboards, and third-party SaaS integrations. While APIs enable rapid product development, they frequently expose underlying data structures directly to the public web.

A read-only API Security Assessment was performed against the **ReqRes API** platform. The evaluation examined endpoint authorization, token hygiene, Cross-Origin Resource Sharing (CORS), and data exposure.

5	1	3	1
ENDPOINTS TESTED	HIGH SEVERITY (BOLA)	MEDIUM SEVERITY	LOW SEVERITY

2. OWASP API Security Findings Matrix

Finding ID	OWASP Category	Risk Title	Severity	CVSS
SEC-API-01	API1:2023 BOLA	Broken Object Level Authorization (Sequential ID Access)	HIGH	7.5

Finding ID	OWASP Category	Risk Title	Severity	CVSS
SEC-API-02	API2:2023 Broken Auth	Opaque Token Generation Lacking Expiry & Signatures	MEDIUM	6.5
SEC-API-03	API3:2023 Property Level	Excessive Data Exposure in User Resource Responses	MEDIUM	5.3
SEC-API-04	API7:2023 Misconfig	Overly Permissive CORS Policy (Access-Control-Allow-Origin: *)	MEDIUM	5.3
SEC-API-05	API7:2023 Misconfig	Infrastructure Information Disclosure via Response Headers	LOW	3.3

3. In-Depth Technical Findings

SEC-API-01: Broken Object Level Authorization (BOLA / IDOR)

HIGH SEVERITY (CVSS 7.5)

OWASP Category: API1:2023 Broken Object Level Authorization

Business Impact: Unauthenticated callers can harvest all registered customer profiles by iterating sequential numerical IDs (e.g., `/api/users/1`, `/api/users/2`), causing mass data exfiltration and violating GDPR data privacy mandates.

```
GET /api/users/2 HTTP/1.1 Host: reqres.in [No Authorization Header Provided] HTTP/1.1 200 OK { "data": { "id": 2, "email": "janet.weaver@reqres.in", "first_name": "Janet", "last_name": "Weaver" } }
```

Remediation: Mandate Bearer token validation on all object-level endpoints. Enforce record ownership verification and replace sequential IDs with non-guessable UUIDv4 strings.

SEC-API-02: Broken Authentication & Weak Token Hygiene

MEDIUM SEVERITY (CVSS 6.5)

OWASP Category: API2:2023 Broken Authentication

Business Impact: The login endpoint generates static string tokens without cryptographic signatures or expiration timestamps. If intercepted, tokens grant indefinite access and are vulnerable to client-side script theft.

```
POST /api/login HTTP/1.1 {"email": "eve.holt@reqres.in", "password": "cityslicka"}
HTTP/1.1 200 OK {"token": "QpwL5tke4Pnpja7X4"} ⟵ [Static, unsigned, non-expiring
token]
```

Remediation: Implement RFC 7519 JSON Web Tokens (JWT) signed with RS256, enforce short expiration windows (15 minutes), and return tokens in `HttpOnly; Secure; SameSite=Strict` cookies.

SEC-API-03: Excessive Data Exposure in User Resource Responses

MEDIUM SEVERITY (CVSS 5.3)

OWASP Category: API3:2023 Broken Object Property Level Authorization

Business Impact: Endpoints return full internal object properties without data transfer object (DTO) filtering, relying on frontend clients to filter fields.

Remediation: Define explicit response Data Transfer Objects (DTOs) and API schemas to ensure only necessary public attributes are serialized to clients.

SEC-API-04: Overly Permissive CORS Policy (Access-Control-Allow-Origin: *)

MEDIUM SEVERITY (CVSS 5.3)

OWASP Category: API7:2023 Security Misconfiguration

Business Impact: Setting wildcard CORS allows malicious third-party websites in an authenticated victim's browser to read API responses across origins.

```
Access-Control-Allow-Origin: *
```

> **Remediation:** Replace wildcard * with an explicit whitelist of trusted frontend origins (e.g., `https://app.reqres.in`).

4. Phased Remediation Roadmap

```
Phase 1: Immediate Sprint (Within 1-2 Weeks) • Require Bearer token authentication across all user endpoints (SEC-API-01) • Restrict CORS Access-Control-Allow-Origin to trusted origin domains (SEC-API-04) Phase 2: Authentication Hardening (Month 1) • Replace static tokens with RS256-signed JWTs with 15-minute expiration (SEC-API-02) • Deliver session tokens via Secure, HttpOnly, SameSite=Strict cookies Phase 3: Data Architecture & Posture (Quarter 1) • Migrate database schemas from sequential integer IDs to UUIDv4 (SEC-API-01) • Implement DTO serialization to prevent excessive data exposure (SEC-API-03) • Enforce API rate limiting (RateLimit-Limit: 100/min per IP/Token)
```

Future Interns — Cyber Security Internship Task 3 Assessor: Harsha H